Programming as a Way of Thinking

Concise theory notes / Core concepts / Exam-ready examples

Central idea

Programming teaches precise, logical problem-solving. Computer scientists express computations in formal languages, design solutions, and test their predictions.

1 Arithmetic operators

An **operator** performs a computation. Its input values are called **operands**.

Operator	Meaning	Example	Value
+	Addition	7 + 3	10
-	Subtraction	7 - 3	4
*	Multiplication	7 * 3	21
/	True division	7 / 2	3.5
//	Floor division	7 // 2	3
**	Exponentiation	2 ** 3	8

- **Division:** / returns a float; // rounds downward (for example, -7 // 2 is -4).
- **Power:** ** means exponentiation. The symbol ^ means bitwise XOR, not power.

2 Expressions and precedence

An **expression** combines values, operators, or function calls and **evaluates to a value**. Python applies precedence: parentheses, exponentiation, multiplication/division, then addition/subtraction.

```
6 + 6 ** 2       # 42
12 + 5 * 6       # 42
(12 + 5) * 6     # 102
```

3 Arithmetic functions

Function	Purpose	Example	Value
round()	Round to nearest value	round(4.6)	5
abs()	Absolute value	abs(-8)	8
type()	Identify type	type(25)	int
len()	Count characters	len("Python")	6

Function call: write the function name and arguments in parentheses. For example, `abs(-8)` is valid; `abs -8` is not a call. At exact half-integers, `round()` uses ties to even: `round(42.5)` is 42; `round(43.5)` is 44.

4 Strings

A **string** (`str`) is a sequence of characters enclosed in straight single or double quotation marks. Spaces, punctuation, and digits are allowed.

```
"Think" + " Python"  # "Think Python"  (concatenation)
"Hi" * 3            # "HiHiHi"       (repetition)
len("Hello")         # 5
```

- **Concatenation:** the + operator joins strings end to end.
- **Repetition:** the * operator copies a string several times.
- **Quoting:** "25" is a string, not a number. Curly quotation marks or backticks are not valid string delimiters.

5 Values, types and conversion

A **value** is a piece of data. A **type** is the category the value belongs to.

Type	Meaning	Example
int	Whole number	25
float	Floating-point number	25.0
str	Sequence of characters	"25"

Convert between compatible types using `int()`, `float()`, and `str()`.

```
int("25")       # 25
float("2.5")    # 2.5
int(3.9)        # 3    (truncates toward zero)
"10" + 5        # TypeError
int("10") + 5   # 15
```

- **Large integers:** `1_000_000` is an integer. `1,000,000` is not a single integer expression in Python.

6 Formal vs natural languages

Natural languages evolve in human communities. **Formal languages** are deliberately designed for specific purposes; programming languages express computations.

Feature	Natural language	Formal language
Ambiguity	Meaning may depend on context	Designed to be unambiguous
Redundancy	Often repetitive	Usually more concise
Literalness	Idioms and metaphors	Exact, rule-governed meaning
Structure	Flexible conventions	Small syntax details matter

7 Bugs, syntax errors and debugging

A **bug** is an error in a program. **Debugging** means locating and correcting bugs. Chapter 1 emphasizes learning from error messages and deliberately experimenting with mistakes.

```
abs 5         # SyntaxError: missing function-call parentheses
"25" / 5       # TypeError: incompatible operand types
```

Exam distinction

A syntax error violates Python's structural rules. A TypeError occurs when an operation receives values of incompatible types. Pay attention to the error type and the expression that caused it.

8 Essential glossary

Term	Definition
Arithmetic operator	Symbol representing an arithmetic computation.
Operand	Value on which an operator acts.
Expression	Combination of values, variables and operators.
Value	A piece of data, such as a number or string.
Type	Category of values, such as int, float or str.
Function	Named sequence of statements performing an operation.
Function call	Expression that runs a function using parentheses.
String	Sequence of characters.
Concatenation	Joining two strings end to end.
Syntax error	Code structure error preventing execution.
Formal language	Language designed for a specific purpose.
Bug / debugging	Program error / process of correcting errors.

9 University exam checklist

Be prepared to explain or demonstrate:

- Difference between `/`, `//` and **; operator precedence.
- Difference between `int`, `float` and `str`; how conversion works.
- String concatenation and repetition; the effects of `len()` and `type()`.
- Formal vs natural languages; meaning of syntax error, bug and debugging.

One-minute memory aid

Operators compute. Expressions produce values. Functions perform tasks. Strings store characters. Types classify values. Formal languages require precision. Debugging finds and fixes bugs.

Next chapter: Variables and Statements. Variables allow you to store and reuse values rather than repeat expressions.